

COMPLETE DEVELOPER GUIDE

CSS Mastery

From Basic Selectors to Pro-Level Architecture —
Every Concept with Examples

Selectors & Specificity

Box Model

Flexbox

Grid

Animations

Custom Properties

Responsive Design

Architecture

4

LEVELS

25+

CONCEPTS

80+

EXAMPLES

Table of Contents

A structured path from CSS fundamentals to professional-grade styling systems

BASIC

Level 1 — Foundations

1. What is CSS? How Browsers Apply Styles
2. CSS Syntax, Selectors & Combinators
3. Specificity, Cascade & Inheritance
4. Colors — All Formats & Functions
5. Typography — Fonts, Text & Web Fonts
6. The Box Model — Margin, Border, Padding, Content
7. Display & Visibility

INTERMEDIATE

Level 2 — Layout & Styling

8. Positioning — Static, Relative, Absolute, Fixed, Sticky
9. Flexbox — Complete Guide
10. CSS Grid — Complete Guide
11. Responsive Design & Media Queries
12. Backgrounds, Gradients & Shadows
13. Borders, Outlines & Border Radius
14. Overflow, Scroll & Clipping

ADVANCED

Level 3 — Advanced CSS

15. Transitions & Animations
16. Transforms — 2D & 3D
17. Pseudo-Classes & Pseudo-Elements
18. CSS Custom Properties (Variables)
19. CSS Functions — calc(), clamp(), min(), max()
20. Filters & Blend Modes

PRO

Level 4 — Professional Mastery

21. CSS Architecture — BEM, SMACSS, Utility-First
22. CSS Logical Properties
23. Container Queries
24. CSS Layers (@layer)
25. Performance & Best Practices

1. What is CSS?

Cascading Style Sheets — the language that makes the web beautiful

CSS (**Cascading Style Sheets**) controls the visual presentation of HTML documents. While HTML defines *structure*, CSS defines *appearance* — colors, fonts, spacing, layout, and animations.

Three Ways to Add CSS

HTML + CSS

```
/* 1. External stylesheet (BEST PRACTICE) */
<link rel="stylesheet" href="styles.css">

/* 2. Internal <style> block (single-page use) */
<style>
  body { background: #f0f0f0; }
</style>

/* 3. Inline styles (avoid – hard to maintain) */
<p style="color: red; font-size: 16px;">Text</p>
```

How the Cascade Works

CSS is "cascading" because multiple stylesheets and rules combine. The browser resolves conflicts using this priority order (highest wins):

Priority	Source	Example
1 (highest)	Inline styles	<code>style="color:red"</code>
2	ID selectors	<code>#header { color: red }</code>
3	Class / pseudo-class	<code>.nav-link:hover</code>
4	Element selectors	<code>p { color: red }</code>
5 (lowest)	Browser defaults	User-agent stylesheet

Basic Syntax

CSS

```
/* selector { property: value; } */

h1 {
  color: #2c5364; /* hex color */
  font-size: 2rem; /* relative unit */
  margin-bottom: 16px; /* absolute unit */
  text-align: center; /* keyword value */
}
```

```
/* Multiple selectors - same styles */  
h1, h2, h3 {  
  font-family: 'Inter', sans-serif;  
}
```

 **Tip:** Always use an external CSS file. It can be cached by the browser, reducing load time on repeat visits. Multiple pages share one file — update once, change everywhere.

2. Selectors & Combinators

Targeting exactly the elements you want to style

CSS — ALL SELECTOR TYPES

```
/* — Basic Selectors — */
*      { box-sizing: border-box; } /* Universal */
p      { color: #333; }           /* Type/element */
.card  { padding: 16px; }         /* Class */
#hero  { height: 100vh; }         /* ID */

/* — Attribute Selectors — */
a[href]      { color: blue; } /* Has attribute */
input[type="email"] { border: 2px solid blue; }
a[href^="https"] { color: green; } /* Starts with */
a[href$=".pdf"]   { color: red; }   /* Ends with */
a[href*="google"] { color: orange; } /* Contains */
div[class~="active"] { display: block; } /* Word in list */

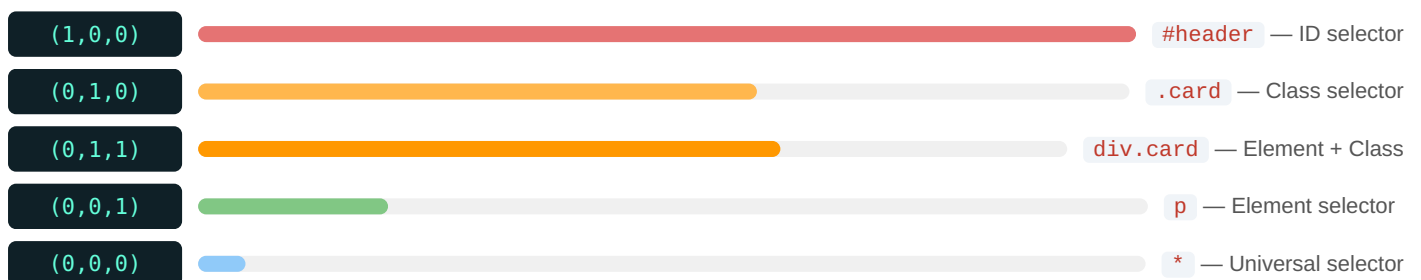
/* — Combinators — */
nav a      { color: white; } /* Descendant (any depth) */
ul > li    { color: blue; }  /* Direct child only */
h2 + p     { font-weight: bold; } /* Adjacent sibling */
h2 ~ p     { color: gray; }   /* General sibling */

/* — Grouping — */
h1, h2, h3 { font-family: sans-serif; }

/* — Chaining (AND logic) — */
p.intro.featured { font-size: 1.2rem; } /* p AND .intro AND .featured */
input:focus:valid { border-color: green; }
```

Specificity — Visual Reference

When two rules target the same element, the one with higher specificity wins. Specificity is calculated as three numbers: **(IDs, Classes, Elements)**.



⚠ **Avoid !important** : It overrides all specificity — even inline styles. It breaks the cascade and makes debugging painful. Use it only as a last resort or for utility classes (e.g., `.hidden { display: none !important; }`).

4. Colors & 5. Typography

All color formats and professional text styling techniques

Every Color Format in CSS

CSS — COLOR FORMATS

```
.colors {  
  /* Named colors (147 built-in names) */  
  color: tomato;  
  color: cornflowerblue;  
  
  /* Hex — #RRGGBB or shorthand #RGB */  
  color: #2c5364;           /* 6-digit */  
  color: #2c536480;         /* 8-digit with alpha */  
  color: #f00;              /* shorthand = #ff0000 */  
  
  /* RGB and RGBA */  
  color: rgb(44, 83, 100);  
  color: rgba(44, 83, 100, 0.5); /* 50% opacity */  
  
  /* Modern: space-separated with optional alpha */  
  color: rgb(44 83 100 / 50%);  
  
  /* HSL — Hue Saturation Lightness */  
  color: hsl(200, 40%, 28%);  
  color: hsl(200 40% 28% / 0.8);  
  
  /* OKLCH — best for color manipulation (modern) */  
  color: oklch(0.55 0.13 220);  
  
  /*currentColor — inherits parent's color */  
  border-color: currentColor;  
  fill: currentColor; /* SVG icons inherit text color */  
  
  /* Transparent */  
  background: transparent;  
}
```

Typography — Complete Reference

CSS — TYPOGRAPHY

```
body {  
  /* Font stack — fallback chain */  
  font-family: 'Inter', 'Segoe UI', system-ui, sans-serif;  
  font-size: 16px;           /* base = 1rem */  
  font-weight: 400;          /* 100-900, or named */  
  font-style: normal;        /* italic | oblique */  
  line-height: 1.6;          /* unitless = preferred */  
  letter-spacing: 0.02em;    /* tracking */  
  word-spacing: 0.05em;
```

```

}

h1 {
  font-size: clamp(1.8rem, 4vw, 3rem); /* responsive */
  font-weight: 700;
  text-transform: uppercase;          /* | capitalize | none */
  text-decoration: underline;
  text-align: center;
}

p {
  text-indent: 2em;
  white-space: nowrap;               /* | pre | pre-wrap | pre-line */
  word-break: break-word;
  overflow-wrap: anywhere;
  hyphens: auto;
}

/* Text overflow ellipsis */
.truncate {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

/* Web fonts from Google Fonts */
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;700&display=swap');

/* Self-hosted web font */
@font-face {
  font-family: 'MyFont';
  src: url('font.woff2') format('woff2'),
       url('font.woff') format('woff');
  font-weight: 400;
  font-display: swap; /* show fallback while loading */
}

```

6. The CSS Box Model

Every element is a rectangular box — understanding this is fundamental to all CSS layout

Every HTML element is composed of four layers from inside to outside: **Content** → **Padding** → **Border** → **Margin**.

CSS — BOX MODEL

```
.box {  
  /* Content */  
  width: 300px;  
  height: 200px;  
  min-width: 200px; max-width: 100%;  
  min-height: 100px; max-height: 500px;  
  
  /* Padding — space INSIDE the border */  
  padding: 20px;           /* all sides */  
  padding: 10px 20px;      /* top/bottom left/right */  
  padding: 5px 10px 15px 20px; /* top right bottom left */  
  padding-top: 10px; padding-right: 20px;  
  
  /* Border */  
  border: 2px solid #2c5364; /* width style color */  
  border-top: 3px dashed red;  
  border-radius: 8px;        /* rounded corners */  
  border-radius: 50%;        /* circle/ellipse */  
  
  /* Margin — space OUTSIDE the border */  
  margin: 20px;             /* all sides */  
  margin: 0 auto;           /* center horizontally */  
  margin-top: auto;         /* push to bottom (in flex) */  
}  
  
/* box-sizing: border-box (USE THIS ALWAYS) */  
/* width INCLUDES padding and border */  
*, *::before, *::after {  
  box-sizing: border-box; /* width = content+padding+border */  
}  
  
/* Without it: content-box = width only content area */  
/* A 300px box + 20px padding = 340px wide — CONFUSING! */  
/* border-box: 300px box + 20px padding = still 300px */
```

Margin Collapse

CSS — MARGIN COLLAPSE

```
/* Adjacent block margins COLLAPSE — only the larger survives */  
h1 { margin-bottom: 30px; }  
p { margin-top: 20px; }  
/* Result: 30px gap (NOT 50px) — bigger margin wins */  
  
/* Collapse does NOT happen in: */
```



```
/* flex/grid containers, floated elements,  
   inline-block, overflow: hidden parents */
```

💡 **Always use `box-sizing: border-box`** in your CSS reset. Add it to `*`, `*::before`, `*::after` at the top of every project. This prevents the most common beginner layout bugs where elements are wider than expected.

8. CSS Positioning

Controlling exactly where elements appear on the page

CSS — ALL POSITION VALUES

```
/* STATIC (default) — normal document flow, top/left/etc ignored */
.box { position: static; }

/* RELATIVE — offset from its NORMAL position, keeps original space */
.nudged {
  position: relative;
  top: 10px;
  left: 20px;
}
/* Also creates a stacking context for absolute children */

/* ABSOLUTE — removed from flow, positioned relative to nearest
   positioned ancestor (relative/absolute/fixed/sticky) */
.parent { position: relative; } /* anchor point */
.badge {
  position: absolute;
  top: -8px;
  right: -8px;
}

/* FIXED — stays in viewport while page scrolls */
.navbar {
  position: fixed;
  top: 0; left: 0;
  width: 100%;
  z-index: 1000;
}

/* STICKY — relative until scroll threshold, then fixed */
.sticky-header {
  position: sticky;
  top: 0; /* sticks when this close to viewport top */
  background: white;
  z-index: 100;
}

/* CENTERING with absolute */
.centered {
  position: absolute;
  top: 50%; left: 50%;
  transform: translate(-50%, -50%);
}

/* Z-INDEX — stacking order (higher = in front) */
/* Only works on positioned elements (not static) */
.modal { z-index: 1000; }
.overlay { z-index: 999; }
```

```
.navbar { z-index: 100; }  
.tooltip { z-index: 50; }
```

Value	In Flow?	Positioned Relative To	Use Case
static	Yes	Normal flow	Default — no special positioning
relative	Yes	Its own normal position	Minor nudges, anchor for children
absolute	No	Nearest positioned ancestor	Tooltips, badges, dropdowns
fixed	No	Viewport	Sticky navbars, floating buttons
sticky	Yes (hybrid)	Scroll container	Section headers, table headers

9. Flexbox — Complete Guide

One-dimensional layout — rows or columns with powerful alignment

CSS — FLEX CONTAINER PROPERTIES

```
.container {  
  display: flex;           /* or inline-flex */  
  
  /* Direction */  
  flex-direction: row;     /* row | row-reverse | column | column-reverse */  
  
  /* Wrapping */  
  flex-wrap: wrap;        /* nowrap | wrap | wrap-reverse */  
  flex-flow: row wrap;    /* shorthand for direction + wrap */  
  
  /* Main axis alignment (horizontal in row direction) */  
  justify-content: flex-start; /* flex-start | flex-end | center */  
  justify-content: space-between; /* equal gaps between items */  
  justify-content: space-around; /* equal space around items */  
  justify-content: space-evenly; /* truly equal gaps */  
  
  /* Cross axis alignment (vertical in row direction) */  
  align-items: center;     /* stretch | flex-start | flex-end | baseline */  
  
  /* Multi-line alignment */  
  align-content: space-between; /* only applies when flex-wrap: wrap */  
  
  /* Gap between items */  
  gap: 16px;              /* row and column gap */  
  gap: 16px 24px;        /* row-gap column-gap */  
}  
  
/* Flex Item Properties */  
.item {  
  /* flex-grow: how much to grow (proportion) */  
  flex-grow: 1;          /* 0 = don't grow (default) */  
  
  /* flex-shrink: how much to shrink */  
  flex-shrink: 1;        /* 0 = don't shrink */  
  
  /* flex-basis: initial size before grow/shrink */  
  flex-basis: 200px;  
  
  /* Shorthand: grow shrink basis */  
  flex: 1 1 0%;          /* equal-width columns */  
  flex: 0 0 auto;        /* fixed size, no grow/shrink */  
  flex: 1;               /* shorthand for "1 1 0%" */  
  
  /* Override container alignment for this item */  
  align-self: flex-end;  
  justify-self: center;  
  
  /* Order (default: 0, lower = earlier) */  
  order: -1;             /* moves to front */  
}
```

```
    order: 2; /* moves to back */
}

/* Perfect centering – the classic use case */
.center-everything {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}
```

10. CSS Grid — Complete Guide

Two-dimensional layout — rows AND columns simultaneously

CSS — GRID CONTAINER & ITEM PROPERTIES

```
.grid {
  display: grid;

  /* Define columns */
  grid-template-columns: 200px 1fr 2fr; /* fixed + flexible */
  grid-template-columns: repeat(3, 1fr); /* 3 equal columns */
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr)); /* responsive! */

  /* Define rows */
  grid-template-rows: 80px auto 60px;
  grid-auto-rows: minmax(100px, auto); /* implicit rows */

  /* Gap */
  gap: 24px;
  row-gap: 16px; column-gap: 24px;

  /* Named areas layout */
  grid-template-areas:
    "header header header"
    "sidebar main main "
    "footer footer footer";

  /* Alignment */
  justify-items: stretch; /* horizontal cell alignment */
  align-items: start; /* vertical cell alignment */
  place-items: center; /* shorthand align + justify */
}

/* Grid Items */
.header { grid-area: header; } /* named areas */
.sidebar { grid-area: sidebar; }
.main { grid-area: main; }

/* Span columns/rows */
.wide {
  grid-column: 1 / 3; /* from line 1 to line 3 */
  grid-column: 1 / -1; /* span full width */
  grid-column: span 2; /* span 2 columns */
  grid-row: span 2;
}
```

 **Flexbox vs Grid:** Use **Flexbox** for one-dimensional UI components (navigation bars, button groups, card rows). Use **Grid** for two-dimensional page layouts (dashboard grids, magazine layouts, photo galleries). They work great together — Grid for the outer layout, Flexbox inside each cell.

11. Responsive Design & Media Queries

Building layouts that work on every screen size

CSS — MEDIA QUERIES

```
/* Mobile-First approach (recommended) */
/* Start with mobile styles, then enhance for larger screens */

.container {
  width: 100%;      /* mobile: full width */
  padding: 0 16px;
}

@media (min-width: 768px) { /* tablet */
  .container { max-width: 720px; margin: 0 auto; }
}

@media (min-width: 1024px) { /* desktop */
  .container { max-width: 1200px; }
}

/* Common breakpoints */
@media (max-width: 480px) { /* small phones */ }
@media (min-width: 576px) { /* landscape phones */ }
@media (min-width: 768px) { /* tablets */ }
@media (min-width: 1024px) { /* desktops */ }
@media (min-width: 1280px) { /* large screens */ }

/* Feature / capability queries */
@media (prefers-color-scheme: dark) {
  body { background: #0f2027; color: #fff; }
}

@media (prefers-reduced-motion: reduce) {
  * { animation-duration: 0.01ms !important; }
}

@media (orientation: landscape) {
  .hero { height: 100vh; }
}

@media (hover: hover) { /* touch vs pointer devices */
  a:hover { text-decoration: underline; }
}

/* CSS Units for responsiveness */
.responsive {
  font-size: 1rem;    /* relative to root font (16px) */
  margin: 2em;        /* relative to element font-size */
  width: 50%;          /* % of parent */
  height: 100vh;       /* viewport height */
  width: 100vw;         /* viewport width */
  padding: 2vmin;       /* smaller of vw/vh */
}
```

```
font-size: clamp(1rem, 2.5vw, 2rem); /* fluid */  
}
```


12. Backgrounds & Gradients

From solid fills to complex multi-layer backgrounds

CSS — BACKGROUNDS

```
.bg-demo {  
  /* Color */  
  background-color: #f0f4f8;  
  
  /* Image */  
  background-image: url('hero.jpg');  
  background-size: cover; /* cover | contain | 100% | auto */  
  background-position: center;  
  background-repeat: no-repeat;  
  background-attachment: fixed; /* parallax effect */  
  background-clip: text; /* clip bg to text */  
  
  /* Multiple backgrounds (layered) */  
  background:  
    url('overlay.png') top center / cover,  
    linear-gradient(rgba(0,0,0,0.4), transparent),  
    url('texture.jpg') center;  
}  
  
/* — Gradients — */  
  
/* Linear gradient */  
.grad-1 {  
  background: linear-gradient(135deg, #0f2027, #2c5364);  
  background: linear-gradient(to right, red, orange, yellow);  
}  
  
/* Radial gradient */  
.grad-2 {  
  background: radial-gradient(circle, #64ffda, #0f2027);  
  background: radial-gradient(ellipse at top, #e66465, #9198e5);  
}  
  
/* Conic gradient */  
.grad-3 {  
  background: conic-gradient(red 0deg, yellow 120deg, blue 240deg);  
}  
  
/* Gradient text */  
.gradient-text {  
  background: linear-gradient(90deg, #64ffda, #2c5364);  
  -webkit-background-clip: text;  
  background-clip: text;  
  -webkit-text-fill-color: transparent;  
  color: transparent;  
}  
  
/* Box Shadow */  
.shadow {
```

```
/* offset-x offset-y blur-radius spread-radius color */
box-shadow: 0 4px 16px 0 rgba(0,0,0,0.12);
box-shadow: inset 0 2px 4px rgba(0,0,0,0.1); /* inset */
/* Layered shadows */
box-shadow:
  0 1px 3px rgba(0,0,0,0.12),
  0 4px 8px rgba(0,0,0,0.08),
  0 16px 32px rgba(0,0,0,0.06);

text-shadow: 2px 2px 4px rgba(0,0,0,0.3);
}
```

15. Transitions & Animations

Bringing interfaces to life with smooth, purposeful motion

CSS — TRANSITIONS

```
.btn {  
  background: #2c5364;  
  color: white;  
  padding: 10px 24px;  
  border-radius: 6px;  
  
  /* transition: property duration timing-function delay */  
  transition: background 0.3s ease;  
  transition: all 0.25s ease-in-out;  
  
  /* Multiple transitions */  
  transition:  
    background 0.3s ease,  
    transform 0.2s ease,  
    box-shadow 0.3s ease;  
}  
  
.btn:hover {  
  background: #0f2027;  
  transform: translateY(-2px);  
  box-shadow: 0 8px 24px rgba(0,0,0,0.2);  
}  
  
/* Timing functions */  
/* ease | linear | ease-in | ease-out | ease-in-out */  
/* cubic-bezier(x1, y1, x2, y2) - custom curve */  
.spring { transition-timing-function: cubic-bezier(0.34,1.56,0.64,1); }
```

CSS — KEYFRAME ANIMATIONS

```
/* 1. Define the animation */  
@keyframes fadeInUp {  
  from {  
    opacity: 0;  
    transform: translateY(30px);  
  }  
  to {  
    opacity: 1;  
    transform: translateY(0);  
  }  
}  
  
@keyframes pulse {  
  0% { transform: scale(1); }  
  50% { transform: scale(1.05); }  
  100% { transform: scale(1); }  
}
```

```
@keyframes spin {
  from { transform: rotate(0deg); }
  to   { transform: rotate(360deg); }
}

/* 2. Apply the animation */
/* animation: name duration timing-fn delay iterations direction */
.hero-text {
  animation: fadeInUp 0.6s ease-out 0.2s both;
}

.loader {
  animation: spin 1s linear infinite;
}

.cta-btn {
  animation: pulse 2s ease-in-out infinite;
}

/* animation-fill-mode: both (stays in final state) */
/* animation-direction: alternate | reverse | alternate-reverse */
/* animation-play-state: paused | running */
.paused:hover { animation-play-state: paused; }
```

16. Transforms & 17. Pseudo-Classes/Elements

Moving, scaling, rotating elements and styling special states

2D & 3D Transforms

```
CSS

.el {
  /* 2D transforms */
  transform: translate(50px, 20px);
  transform: translateX(100px);
  transform: translateY(-20px);
  transform: scale(1.5);
  transform: scaleX(2);
  transform: rotate(45deg);
  transform: skew(10deg, 5deg);

  /* Chained transforms */
  transform:
    translateY(-10px)
    scale(1.1)
    rotate(5deg);

  /* 3D transforms */
  transform: rotateX(45deg);
  transform: rotateY(180deg);
  transform: perspective(400px)
    rotateY(30deg);

  /* Transform origin */
  transform-origin: top left;
  transform-origin: 50% 50%;
}

/* 3D card flip */
.card-3d {
  perspective: 1000px;
}

.card-inner {
  transform-style: preserve-3d;
  transition: transform 0.6s;
}

.card-3d:hover .card-inner {
  transform: rotateY(180deg);
}
```

Pseudo-Classes & Elements

```
CSS

/* Structural */
li:first-child { }
li:last-child { }
li:nth-child(2) { }
li:nth-child(odd) { }
li:nth-child(3n) { }
p:only-child { }
div:empty { }

/* User actions */
a:link { } /* unvisited */
a:visited { }
a:hover { }
a:focus { }
a:active { } /* during click */
:focus-visible { } /* keyboard only */

/* Form states */
input:required { }
input:valid { }
input:invalid { }
input:disabled { }
input:checked { }
input:placeholder-shown {}

/* Negation & "is" */
:not(.disabled) { }
:is(h1, h2, h3) { }
:where(.card, .panel) {}
:has(img) { } /* parent selector! */

/* Pseudo-elements (content generators) */
p::before {
  content: "→ ";
  color: #2c5364;
}
p::after { content: " ←"; }
::first-line { font-weight: bold; }
::selection { background: #64ffda; }
::placeholder { color: #999; }
::marker { color: red; }
```

18. CSS Custom Properties & 19. Functions

Variables, dynamic values, and powerful mathematical functions

CSS — CUSTOM PROPERTIES (VARIABLES)

```
/* Define variables in :root = global scope */
:root {
  /* Design tokens */
  --color-primary: #2c5364;
  --color-accent: #64ffda;
  --color-text: #1a1a2e;
  --color-bg: #ffffff;

  --font-sans: 'Inter', system-ui, sans-serif;
  --font-size-base: 16px;
  --font-size-lg: 1.25rem;
  --font-size-sm: 0.875rem;

  --spacing-xs: 4px;
  --spacing-sm: 8px;
  --spacing-md: 16px;
  --spacing-lg: 32px;

  --radius: 8px;
  --shadow: 0 4px 16px rgba(0,0,0,0.1);
  --transition: all 0.25s ease;
}

/* Dark mode theme overrides */
@media (prefers-color-scheme: dark) {
  :root {
    --color-text: #e2e8f0;
    --color-bg: #0f2027;
  }
}

/* Using variables */
.card {
  background: var(--color-bg);
  color: var(--color-text);
  font-family: var(--font-sans);
  padding: var(--spacing-md);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  transition: var(--transition);
}

/* Default fallback values */
.el { color: var(--maybe-undefined, #333); }

/* Component-level override */
.btn-danger {
```

```
--color-primary: #e74c3c; /* scoped to this component */  
}
```

CSS Math Functions

CSS — FUNCTIONS

```
.fluid {  
  /* calc() — mix units */  
  width:   calc(100% - 240px);  
  margin:  calc(var(--spacing-md) * 2);  
  padding: calc(1rem + 2vw);  
  
  /* min() — use smallest value */  
  width: min(90%, 800px); /* never wider than 800px */  
  
  /* max() — use largest value */  
  width: max(300px, 50%); /* never narrower than 300px */  
  
  /* clamp(min, preferred, max) — fluid typography! */  
  font-size: clamp(1rem, 2.5vw, 2.5rem);  
  padding:   clamp(16px, 4vw, 64px);  
  
  /* abs, round, mod, rem (newer) */  
  width: round(10.7px, 1px); /* = 11px */  
}
```

20. Filters & Blend Modes

Visual effects without JavaScript or image editing software

CSS — FILTERS & BLEND MODES

```
/* filter – applies to the element and its children */
.filters {
  filter: blur(4px);
  filter: brightness(0.8);      /* 0=black, 1=normal, 2=bright */
  filter: contrast(1.5);
  filter: grayscale(100%);
  filter: sepia(80%);
  filter: saturate(2);
  filter: hue-rotate(90deg);
  filter: invert(100%);
  filter: opacity(0.5);
  filter: drop-shadow(4px 4px 8px rgba(0,0,0,0.3));

  /* Chain multiple filters */
  filter: brightness(0.9) contrast(1.1) saturate(1.3);
}

/* backdrop-filter – blurs content BEHIND element */
.glass-card {
  background: rgba(255,255,255,0.2);
  backdrop-filter: blur(10px) saturate(1.5);
  border: 1px solid rgba(255,255,255,0.3);
  /* "Glassmorphism" effect */
}

/* Mix Blend Mode – how element blends with content below */
.blend {
  mix-blend-mode: multiply;
  mix-blend-mode: screen;      /* lightens */
  mix-blend-mode: overlay;     /* contrast boost */
  mix-blend-mode: difference;  /* inverted contrast */
  mix-blend-mode: luminosity;
  mix-blend-mode: color;
}

/* Background blend mode */
.bg-blend {
  background-image: url('photo.jpg'),
                    linear-gradient(blue, red);
  background-blend-mode: luminosity; /* blends the two BGs */
}

/* Clipping & Masking */
.clip {
  clip-path: circle(50%);
  clip-path: polygon(0 0, 100% 0, 100% 80%, 0 100%);
  clip-path: ellipse(60% 40% at 50% 50%);
}
```



```
clip-path: inset(10px 20px round 8px);
```

```
}
```

21. CSS Architecture — BEM & Patterns

Organizing CSS at scale to keep codebases maintainable

BEM — Block Element Modifier

CSS + HTML — BEM NAMING

```
/* BEM: Block__Element--Modifier */
/* Block = standalone component: .card, .btn, .nav */
/* Element = part of block: .card__title, .btn__icon */
/* Modifier = variant: .card--featured, .btn--large */

/* CSS */
.card { background: white; }
.card__image { width: 100%; }
.card__title { font-size: 1.2rem; }
.card__body { padding: 16px; }
.card--featured { border: 2px solid gold; }
.card--dark { background: #333; }

/* HTML */
/* <div class="card card--featured"> */
/*   <img class="card__image" ...> */
/*   <div class="card__body"> */
/*     <h3 class="card__title">... */
/*   </div> */
/* </div> */

/* Utility classes (Tailwind approach) */
.flex { display: flex; }
.items-center { align-items: center; }
.justify-between { justify-content: space-between; }
.p-4 { padding: 1rem; }
.text-center { text-align: center; }
.hidden { display: none !important; }
```

Container Queries (Pro)

CSS — @CONTAINER

```
/* Container queries — component responds to ITS container width */
/* NOT the viewport! Much more flexible than media queries. */
.card-wrapper {
  container-type: inline-size; /* enable container query */
  container-name: card; /* optional name */
}

@container card (min-width: 400px) {
  .card { display: grid; grid-template-columns: 1fr 2fr; }
}

@container (max-width: 300px) {
```

```
.card__title { font-size: 0.9rem; }  
}
```

CSS @layer (Cascade Layers)

CSS — @LAYER

```
/* Define layer order — later layers WIN */  
@layer reset, base, components, utilities;  
  
@layer reset {  
  *, *::before, *::after { box-sizing: border-box; }  
}  
@layer base {  
  body { font-family: sans-serif; }  
}  
@layer components {  
  .btn { padding: 8px 16px; }  
}  
@layer utilities {  
  .hidden { display: none; } /* overrides components */  
}
```

22. CSS Logical Properties & Modern Reset

Writing direction-aware CSS and the modern baseline reset

Logical Properties

Replace physical directions (left/right) with logical ones that adapt to different writing modes (LTR, RTL, vertical).

CSS

```
.box {  
  /* Inline = horizontal (LTR/RTL) */  
  margin-inline-start:  
16px; /* =margin-left */  
  margin-inline-end: 16px; /* =margin-right */  
  margin-inline: auto; /* center! */  
  
  /* Block = vertical */  
  margin-block-start: 24px; /* =margin-top */  
  margin-block-end: 24px; /* =margin-bottom */  
  margin-block: 16px;  
  
  padding-inline: 20px;  
  padding-block: 12px;  
  
  /* Dimensions */  
  inline-size: 300px; /* =width */  
  block-size: 200px; /* =height */  
  
  /* Borders */  
  border-inline-start: 4px solid blue;  
  border-block-end: 1px solid #ccc;  
}
```

Modern CSS Reset

CSS RESET

```
/* Modern CSS Reset */  
*, *::before, *::after {  
  box-sizing: border-box;  
}  
  
* {  
  margin: 0;  
  padding: 0;  
}  
  
html {  
  -webkit-text-size-adjust: none;  
  hanging-punctuation: first last;  
}  
  
body {  
  min-height: 100dvh;  
  line-height: 1.5;  
  -webkit-font-smoothing: antialiased;  
}  
  
img, picture, video, canvas, svg {  
  display: block;  
  max-width: 100%;  
}  
  
input, button, textarea, select {  
  font: inherit;  
}  
  
p, h1, h2, h3, h4, h5, h6 {  
  overflow-wrap: break-word;  
}  
  
:focus-visible {  
  outline: 3px solid var(--color-accent);  
  outline-offset: 2px;  
}
```

25. Performance & Pro Cheat Sheet

Optimizing CSS for speed and the most important rules

CSS Performance Rules

CSS — PERFORMANCE PATTERNS

```
/* Use will-change for animated elements */
.animated {
  will-change: transform, opacity;
  /* Promotes to its own GPU layer */
  /* Remove after animation ends to free memory */
}

/* ONLY animate transform and opacity */
/* These run on GPU – no layout recalculation */
.good-animation {
  transition: transform 0.3s, opacity 0.3s;
}
.bad-animation {
  transition: width 0.3s, height 0.3s;
  /* Triggers layout recalculation – SLOW! */
}

/* Contain – limit reflow area */
.widget {
  contain: layout; /* layout changes don't affect outside */
  contain: strict; /* most aggressive containment */
}

/* content-visibility – skip off-screen rendering */
.article {
  content-visibility: auto;
  contain-intrinsic-size: auto 200px;
}

/* Load CSS efficiently */
/* Critical CSS inline in <head>, rest deferred */
/* <link rel="preload" href="styles.css" as="style"> */
```

The Golden Rules of Professional CSS

✓ Always Do

- Use `box-sizing: border-box` globally
- CSS Custom Properties for tokens
- Mobile-first media queries
- `gap` instead of margin hacks in flex/grid
- `clamp()` for fluid typography
- Semantic class names (BEM)

✗ Avoid

- Overusing `!important`
- Deeply nested selectors (3+ levels)
- Animating `width`, `height`, `top`, `left`
- Inline styles (hard to override/maintain)
- ID selectors for styling
- Tables for layout

- `:focus-visible` for accessibility
- Only animate `transform` & `opacity`

- `outline: none` (breaks keyboard nav)
- Removing `:focus` styles entirely

Complete Properties Quick Reference

Category	Key Properties
Typography	<code>font-family</code> <code>font-size</code> <code>font-weight</code> <code>line-height</code> <code>letter-spacing</code> <code>text-align</code> <code>text-transform</code> <code>text-overflow</code> <code>white-space</code>
Box Model	<code>width</code> <code>height</code> <code>min/max-width/height</code> <code>margin</code> <code>padding</code> <code>border</code> <code>border-radius</code> <code>box-sizing</code> <code>overflow</code>
Layout	<code>display</code> <code>position</code> <code>top</code> <code>right</code> <code>bottom</code> <code>left</code> <code>z-index</code> <code>float</code> <code>clear</code>
Flexbox	<code>flex-direction</code> <code>flex-wrap</code> <code>justify-content</code> <code>align-items</code> <code>align-content</code> <code>gap</code> <code>flex</code> <code>order</code> <code>align-self</code>
Grid	<code>grid-template-columns/rows</code> <code>grid-area</code> <code>grid-column/row</code> <code>gap</code> <code>auto-fit</code> <code>minmax</code> <code>place-items</code>
Visual	<code>background</code> <code>color</code> <code>opacity</code> <code>border-radius</code> <code>box-shadow</code> <code>filter</code> <code>backdrop-filter</code> <code>clip-path</code>
Animation	<code>transition</code> <code>animation</code> <code>@keyframes</code> <code>transform</code> <code>will-change</code> <code>animation-play-state</code>
Modern	<code>var()</code> <code>calc()</code> <code>clamp()</code> <code>min()</code> <code>max()</code> <code>@layer</code> <code>@container</code> <code>@media</code> <code>container-type</code>



You've completed the CSS Basic to Pro Guide

Next steps: [Sass/SCSS preprocessor](#) → [Tailwind CSS](#) → [CSS-in-JS](#) → [PostCSS](#) → [Design Systems](#)